Análise Comparativa de Escalonadores de Processos em Sistemas Operacionais para Servidores Críticos: CentOS, RHEL e Slackware

Vitor Hugo Altmann
Ciência da Computação
Universidade Comunitária da Região de Chapecó (Unochapecó)
Chapecó, Santa Catarina, Brasil

***Resumo*—A eficiência de um servidor crítico em ambientes corporativos depende diretamente da capacidade do seu Sistema Operacional em gerenciar os recursos de hardware. O coração deste gerenciamento é o escalonador de processos (scheduler), responsável por distribuir o tempo de CPU entre as tarefas de forma justa e otimizada. Este artigo apresenta uma análise profunda dos mecanismos de escalonamento presentes em três distribuições Linux consolidadas em servidores point release: Red Hat Enterprise Linux (RHEL), CentOS e Slackware. Discutimos o funcionamento de baixo nível do Completely Fair Scheduler (CFS), sua interação com arquiteturas de hardware modernas, os requisitos mínimos e os cenários ideais de uso para cada sistema.**

***Palavras-chave*—Sistemas Operacionais, Escalonadores, CFS, RHEL, CentOS, Slackware, Hyper-Threading.**

I. Introdução

No contexto de infraestrutura de TI corporativa, a escolha de um Sistema Operacional (SO) transcende a preferência por interfaces ou gerenciadores de pacotes; ela reside na estabilidade, previsibilidade e no desempenho algorítmico do kernel. Sistemas *point release* — que recebem atualizações massivas em versões específicas em vez de *rolling releases* constantes — são preferidos em ambientes de missão crítica devido à sua imutabilidade arquitetônica ao longo de ciclos de vida de até 10 anos.

O escalonador de processos é a estrutura de controle que decide qual *thread* ou processo será executado na Unidade Central de Processamento (CPU) em um dado instante. Em sistemas que processam pesadas cargas de dados, como plataformas ERP financeiras e pipelines analíticos de alta complexidade, o tempo de resposta e o *throughput* geral são determinados pelas decisões de nanossegundos deste componente.

Este trabalho foca na arquitetura interna do escalonador adotado pelo núcleo Linux e nas distinções de implementação e sintonia fina entre Red Hat Enterprise Linux (RHEL), CentOS e Slackware.

II. Fundamentação Teórica: O Escalonador

A. O Que É e Por Que Importa?: O escalonamento é a base da multiprogramação. Em sistemas modernos, há invariavelmente mais processos ativos do que núcleos físicos (ou *threads* lógicas, no caso de processadores multithread) disponíveis. A necessidade primária é evitar a inanição (*starvation*) de processos e maximizar a taxa de utilização da CPU.

O escalonador não atua de forma isolada; ele opera manipulando contextos de registradores na memória, efetuando o chaveamento de contexto (*context switch*). Um modelo de escalonamento mal planejado pode resultar em latências catastróficas para aplicações, corrupção em fluxos de dados concorrentes ou desperdício de ciclos em arquiteturas NUMA (*Non-Uniform Memory Access*) devido a transferências dispendiosas através do barramento inter-nós.

B. O Completely Fair Scheduler (CFS): Como RHEL, CentOS e Slackware são distribuições baseadas no kernel Linux, todos compartilham a mesma fundação algorítmica dominante desde o kernel 2.6.23: o *Completely Fair Scheduler* (CFS).

O funcionamento do CFS diverge radicalmente dos escalonadores clássicos $O(1)$ baseados em matrizes estáticas de prioridade. Ele modela computacionalmente uma "CPU ideal e perfeitamente multitarefa", onde cada processo ganharia uma fração exata e contínua do poder de processamento proporcional ao seu peso lógico.

Na impossibilidade física de rodar tudo ao mesmo tempo, o CFS rastreia de forma implacável um *vruntime* (tempo de execução virtual) de cada processo em nível de nanossegundos. A estrutura de dados subjacente utilizada para enfileirar processos não é uma lista convencional, mas sim uma árvore rubro-negra (*Red-Black Tree*).

- **Complexidade Matemática:** A inserção e remoção de processos na árvore rubro-negra possuem complexidade assintótica $O(\log N)$. O nó situado mais à esquerda da árvore sempre abriga o processo com o menor *vruntime*, ou seja, o algoritmo rapidamente aponta para o processo que mais "precisa" de CPU no momento.
- **Tratamento de Topologias Físicas:** O CFS é construído para decifrar a topologia do hardware nativo (*Sched Domains*). Se o processador suporta *Hyper-Threading* em arquiteturas baseadas em x86_64, o CFS agrupa essas *threads* em domínios unificados, inibindo a migração de um processo pesado em cache para outro núcleo físico caso os dados ainda estejam aquecidos no cache L2 ou L3.

III. Análise Comparativa dos Sistemas

Apesar do algoritmo central base ser o mesmo, a implementação no *bare metal* e as decisões de projeto destas três distribuições alteram violentamente a resposta do *scheduler* em produção, seja através de políticas de compilação, extensões modulares ou de *tuning* dinâmico.

A. Red Hat Enterprise Linux (RHEL): O RHEL consolida-se como o padrão corporativo na indústria de infraestrutura. Seu kernel base é rigidamente homologado e focado em confiabilidade sistêmica.

Como o Escalonador atua: A Red Hat não entrega o kernel Linux puramente *vanilla*. Ela introduziu e amadureceu o *Tuned*, um daemon inteligente de ajuste de perfis e de parâmetros do kernel. O RHEL permite alternar o comportamento interno do CFS sob demanda. Em perfis corporativos de banco de dados e plataformas financeiras, parâmetros sensíveis de escalonamento (como *sched_migration_cost_ns*) são inflados agressivamente para favorecer o *throughput* em detrimento da latência interativa, forçando a CPU a aguentar explosões prolongadas de I/O de disco.

Requisitos Mínimos Recomendados (RHEL 9):

- 1.5 GB de RAM.
- 20 GB de armazenamento persistente.
- Processadores x86_64 estendidos, ARM64 ou equivalentes corporativos compatíveis.

Onde usar: Data Centers corporativos. Servidores de ERPs empresariais monolíticos, ecossistemas de processamento de alto volume (SAP, Oracle) e ambientes que exigem SLA de missão crítica com responsabilidade jurídica.

B. CentOS (Linhagem Tradicional): Em suas versões *point release* autênticas (como o icônico CentOS 7), este sistema construiu sua reputação como um clone binário direto do RHEL, entregando o *core* corporativo da Red Hat sem as licenças e chancelas comerciais.

Como o Escalonador atua: Seu comportamento de escalonamento espelha o RHEL com exatidão. O grande diferencial arquitetônico do CentOS tradicional recai sobre a política de *backports* no kernel. Em vez de subir a versão geral do núcleo e quebrar a retrocompatibilidade (ABI), os mantenedores injetam correções estruturais refinadas escritas em C e Assembly nativo para otimizar alocação de memória NUMA nas antigas raízes do kernel padrão, mantendo estabilidade cega e previsibilidade absoluta.

Requisitos Mínimos (CentOS 7):

- 1 GB de RAM (1.5 GB recomendado).
- 10 GB de disco.
- Processador x86_64 (exclusividade em 64 bits em versões modernas).

Onde usar: Hospedagem de infraestruturas *on-premise* resilientes, *clusters* de computação distribuída locais e domínios em que a arquitetura deve permanecer paralisada no tempo sem instabilidades, mantendo eficiência robusta e zero custo por nó de execução.

C. Slackware Linux: Sendo a distribuição mais antiga ainda ativamente mantida, o Slackware fundamenta-se sob a diretriz UNIX estrita e o princípio *KISS* (Keep It Simple, Stupid). O sistema recusa camadas ocultas de automação e empodera diretamente o administrador com a visão de mais baixo nível.

Como o Escalonador atua: Diferente da camada engessada da Red Hat, o Slackware embarca kernels o mais próximos possíveis de suas matrizes originais sem modificações obscuras ou de código fechado. Não existem serviços pesados de *tuning* em *background* que interceptam prioridades aleatoriamente. Qualquer injeção de performance no CFS é realizada puramente via interfaces de chamadas no *sysctl.conf*. Esta pureza resulta em um estado latente incrivelmente previsível, onde o escalonador lida majoritariamente apenas com as exatas chamadas de rotina que o administrador projetou a mão em *scripts* procedurais puros.

Requisitos Mínimos (Slackware 15):

- CPU série 486 (x86/x86_64).
- 64 MB de RAM apenas para o *boot* (recomenda-se 1 GB+ para tarefas modernas).
- 5 GB de espaço.

Onde usar: Em instâncias onde se faz necessário o controle cirúrgico de alocação, desenvolvimento dedicado e terminais estritos onde o SO deve operar de forma imutável durante anos imune a microsserviços pesados.

IV. Conclusão

Embora a alma matemática seja idêntica — uma formidável árvore Red-Black processando lógicas na magnitude temporal do nanossegundo ($O(\log N)$) —, a decisão entre RHEL, CentOS e Slackware para infraestruturas críticas transcende o algoritmo. É uma decisão de projeto e governança corporativa. O RHEL e o CentOS manipulam agressivamente o *Completely Fair Scheduler* em favor do *throughput* voltado à aplicações bancárias, enquanto o Slackware sustenta a arquitetura original garantindo previsibilidade absoluta ditada pelas mãos do engenheiro.